Computer Vision Object Detection & Violation Identification System Using YOLOv8

1. Project Overview

I built an end-to-end Computer Vision object detection system using YOLOv8 to identify target objects and potential violations in image feeds. The primary goal was to execute a complete AI development workflow—spanning problem definition, dataset curation, manual annotation verification, YOLO dataset formatting, model training, performance evaluation, and inference on unseen test data. The finalized system detects target classes and renders bounding boxes along with prediction confidence scores on unseen images.

2. Tools and Technologies Used

- Programming Language & Environment: Python 3, Google Colab (Tesla T4 GPU)
- Computer Vision & ML Libraries: Ultralytics YOLOv8, PyTorch, OpenCV
- Annotation & Dataset Tools: Roboflow Annotate
- Data Visualization & Processing: Matplotlib, OS, Shutil

3. Dataset Preparation

- Source: Roboflow Universe (`violation-atfhu` project).
- Dataset Volume: 200+ images.
- Annotations: Manually reviewed and adjusted bounding box coordinates. Exported labels in normalized YOLO format (`.txt` files containing `<class_id> <x_center> <y_center> <width> <height>`).
- Data Split: 70% Training, 20% Validation, 10% Testing.

4. Model Training

- Model Architecture: YOLOv8 Nano (`yolov8n.pt` pretrained weights).
- Hyperparameters:
  Epochs: 30
  Image Size: 640x640
  Batch Size: 8
- Execution Environment:Google Colab T4 GPU acceleration.
- Checkpointing: Best weights file saved to `models/best.pt`.

5. Evaluation Results

- Evaluation Metrics: Precision, Recall, mAP@0.5, and mAP@0.5:0.95.

- Performance Summary: The model converged steadily over 30 epochs, showing low bounding box loss and classification loss on validation data. Precision and recall metrics demonstrated accurate target detection and tight bounding box localized fits.

6. Inference Results

Inference was run on unseen test images using `best.pt` at a confidence threshold of `0.35`. The model successfully detected instances of target classes and generated annotated outputs featuring bounding boxes, predicted labels, and confidence metrics.

7. Error Analysis

Running error analysis on test images from: /content/violation-1/test/images

	Image	Actual Object	Model Prediction	Error Type	Possible Reason
0	camera1_48b02d6010d9_2025_04_14_17_39_30_jpg.r...	no_seatbelt	no_seatbelt (0.47)	Low Confidence	Similar background texture or blurry contours
1	camera1_48b02d6010d9_2025_04_14_17_43_22_jpg.r...	seatbelt	seatbelt (0.46)	Low Confidence	Similar background texture or blurry contours
2	camera1_48b02d6010d9_2025_04_14_17_43_22_jpg.r...	no_seatbelt	no_seatbelt (0.46)	Low Confidence	Similar background texture or blurry contours

3	camera1_48b02d6010d9_2025_04_14_17_43_22_jpg.r...	seatbelt	seatbelt (0.37)	Low Confidence	Similar background texture or blurry contours
4	camera1_48b02d6010d9_2025_04_14_17_53_05_jpg.r...	no_seatbelt	no_seatbelt (0.41)	Low Confidence	Similar background texture or blurry contours
5	camera1_48b02d6010d9_2025_04_14_18_01_22_jpg.r...	no_seatbelt	no_seatbelt (0.40)	Low Confidence	Similar background texture or blurry contours
6	camera2_48b02d6010d9_2025_04_14_17_43_22_jpg.r...	vehicle	vehicle (0.36)	Low Confidence	Similar background texture or blurry contours
7	camera2_48b02d6010d9_2025_04_14_17_59_42_jpg.r...	no_seatbelt	no_seatbelt (0.49)	Low Confidence	Similar background texture or blurry contours
8	camera2_48b02d6010d9_2025_04_14_17_59_42_jpg.r...	seatbelt	seatbelt (0.47)	Low Confidence	Similar background texture or blurry contours
9	camera2_48b02d6010d9_2025_04_14_17_59_42_jpg.r...	seatbelt	seatbelt (0.38)	Low Confidence	Similar background texture or blurry contours

Key Findings: Occluded objects and small instances contributed to false negatives. Adding additional training samples captured under varied lighting conditions will resolve misclassifications.

8. What I Learned

- Executed the complete end-to-end lifecycle of a real-world Computer Vision project.
- Learned manual annotation techniques and normalized YOLO format structures.
- Fine-tuned pretrained YOLOv8 models on GPU hardware using Ultralytics.
- Evaluated models using Precision, Recall, mAP scores, and confusion matrices.
- Analyzed failure modes to outline dataset quality improvements.

9. Future Improvements

- Expand the dataset to 500+ images for greater class balance.
- Experiment with larger YOLO model variants (e.g., YOLOv8s or YOLOv8m).
- Deploy the model as an interactive web demo using Streamlit or FastAPI.